Software Design for *Code Venture*

Monash University Malaysia

Prepared by:

Sahab Ferdous

Taraaf Tazeez Khalidi

Nicolas Kai Xian Weinbrecht

Wong Yew Yi

Date:

21/08/2023

Table of Contents

1. Comprehensive System Context Analysis..... 3
1.1 Operational Context..... 3
1.2 Behavioural Context..... 4
1.3 Ecological Context..... 4
1.4 Sociocultural Context..... 4
2. System Architecture Diagram..... 5
2.1 Layered Architecture..... 5
2.1.1 Data Access Layer (DAL)..... 5
2.1.2 Business Logic Layer (BLL)..... 5
2.1.3 Application Layer..... 5
2.1.4 Presentation Layer..... 5
2.2 Non-Functional Requirements and Trade-offs..... 7
2.2.1 Performance..... 7
2.2.2 Usability..... 7
2.2.3 Reliability..... 7
2.2.4 Security..... 7
2.2.5 Availability..... 7
2.2.6 Maintainability..... 8
2.2.7 Scalability..... 8
2.2.8 Ethics and Inclusivity..... 8
2.2.9 Compatibility..... 8
3. Class Diagram..... 9
3.1 Impact of the system architecture decision..... 10
4. Use Case Diagrams..... 11
4.1 Learner..... 11
4.2 Educator..... 12
4.3 Administrator..... 13
4.4 Parent/Guardian..... 14
5. Sequence Diagrams..... 15

1. Comprehensive System Context Analysis

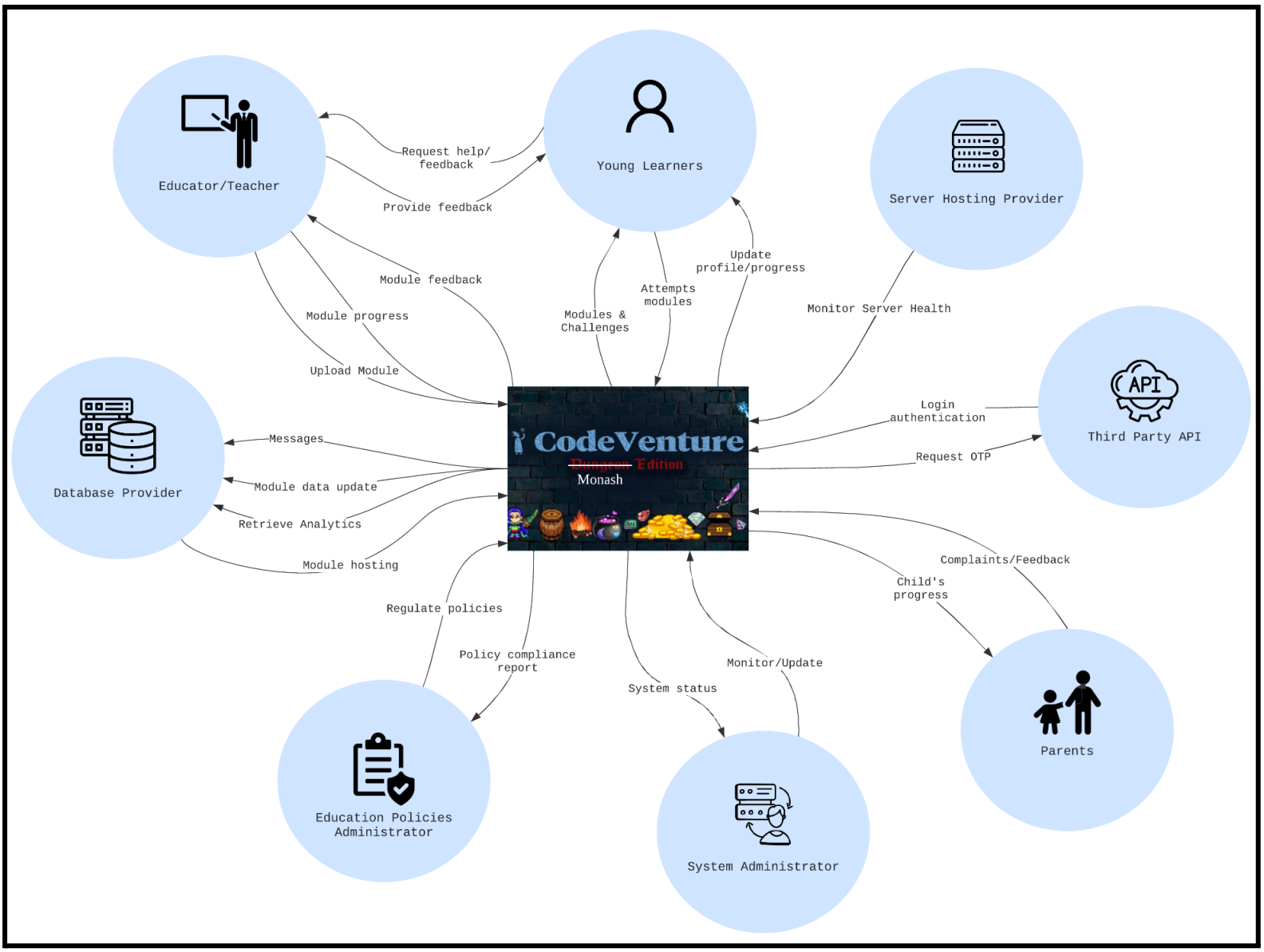


Figure 1.0 - System context diagram

1.1 Operational Context

CodeVenture, being an interactive game platform, necessitates seamless operation across various devices and platforms. The users – primarily young learners, educators, and parents – expect uninterrupted access to learning modules, progress tracking, and other features, be it on a personal computer, tablet, or smartphone. The operational context is fundamentally built around ease of use, adaptability, and efficient content delivery.

1.2 Behavioural Context

The behaviour of CodeVenture's users shapes the dynamics of the platform. Young learners are anticipated to frequently interact with the platform, attempting challenges, quizzes, and progressing through learning modules. Educators might utilise the platform for teaching purposes, assigning tasks, and monitoring students' progress. Meanwhile, parents would be keen to observe their children's growth trajectory and provide feedback when necessary. CodeVenture must cater to these varying behaviours, ensuring a personalised and intuitive experience for each user group.

1.3 Ecological Context

CodeVenture, as a digital platform, holds a commitment towards sustainability. All server operations aim to minimise carbon footprint, and the platform encourages paperless learning. As an e-learning solution, it indirectly contributes to reducing the carbon emissions that traditional brick-and-mortar institutions might generate through daily commutes or resource consumption.

1.4 Sociocultural Context

In today's globalised world, the cultural and social backgrounds of users are diverse. CodeVenture is not just an educational tool; it is a bridge that connects cultures through the universal language of coding. The platform acknowledges global diversities and ensures content is culturally sensitive, promotes inclusivity, and upholds ethical values, making it accessible and respectful to users from different sociocultural backgrounds.

2. System Architecture Diagram

2.1 Layered Architecture

2.1.1 Data Access Layer (DAL)

- **Responsible for** interacting directly with databases.
- **Components:** Database servers, ORM (Object-Relational Mapping) tools, Data models.
- **Rationale:** Centralises data operations, ensuring data integrity and consistency.

2.1.2 Business Logic Layer (BLL)

- **Responsible for** implementing business rules, algorithms, and core platform functionalities.
- **Components:** Learning modules, Progress tracking algorithms, Quiz and challenge logic.
- **Rationale:** Separates core logic from data and presentation, enhancing modularity.

2.1.3 Application Layer

- **Responsible for** managing user sessions, requests, and responses.
- **Components:** User authentication, Profile management, Session controllers.
- **Rationale:** Facilitates communication between the business logic and presentation layers.

2.1.4 Presentation Layer

- **Responsible for** displaying information and interacting with end-users.
- **Components:** User interface, Responsive design elements, Visual feedback components.
- **Rationale:** Provides a dedicated layer for user interaction, ensuring a consistent and pleasant user experience.

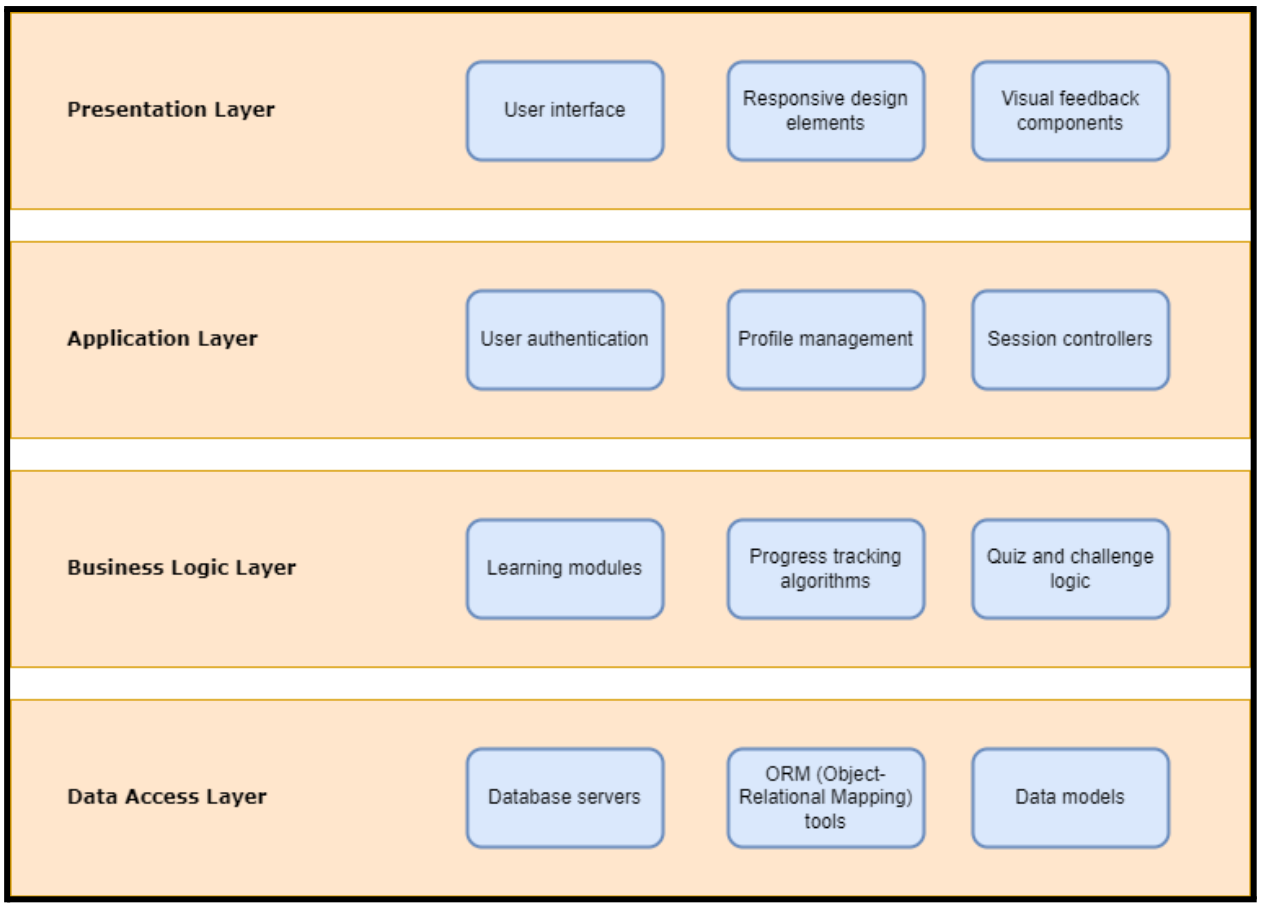


Figure 2.0 – System Architecture Diagram

The Layered Architecture was chosen due to its **modularity** and **clear separation** of concerns, catering to both functional and non-functional requirements of CodeVenture. By compartmentalising tasks within distinct layers, this architecture style streamlines development, troubleshooting, and feature enhancement processes. Moreover, it offers flexibility, facilitating easier adaptation to evolving requirements.

2.2 Non-Functional Requirements and Trade-offs

2.2.1 Performance

- **Decision:** Employ caching mechanisms in the Application Layer for frequently accessed data.
- **Trade-offs:** Slightly increased memory usage but faster data retrieval and response times.

2.2.2 Usability

- **Decision:** Design the Presentation Layer with intuitive UI/UX components.
- **Trade-offs:** Requires additional design iterations but results in improved user engagement.

2.2.3 Reliability

- **Decision:** Implement redundant servers and backup mechanisms in the DAL.
- **Trade-offs:** Increased infrastructure costs but ensures data availability and minimises downtimes.

2.2.4 Security

- **Decision:** Implement robust encryption mechanisms in DAL and secure session management in the Application Layer.
- **Trade-offs:** Might add minimal latency but ensures data protection and user privacy.

2.2.5 Availability

- **Decision:** Use cloud-based solutions with auto-scaling and load balancing for DAL and BLL.
- **Trade-offs:** Higher operational costs but guarantees service availability during peak loads.

2.2.6 Maintainability

- **Decision:** Use a modular approach in the BLL, allowing for easier updates and maintenance.
- **Trade-offs:** Longer initial development phase but facilitates easier long-term management.

2.2.7 Scalability

- **Decision:** The layered approach inherently supports scalability, especially with cloud integrations.
- **Trade-offs:** Potential for slight initial performance overhead but ensures the platform can handle growth.

2.2.8 Ethics and Inclusivity

- **Decision:** Incorporate feedback mechanisms in the Presentation Layer and employ diverse content creation teams for BLL.
- **Trade-offs:** Increased content vetting time but ensures a diverse and inclusive platform.

2.2.9 Compatibility

- **Decision:** Design the Presentation Layer to be responsive and compatible across devices.
- **Trade-offs:** Extends design and testing phases but ensures a wide user base can access the platform seamlessly.

3. Class Diagram

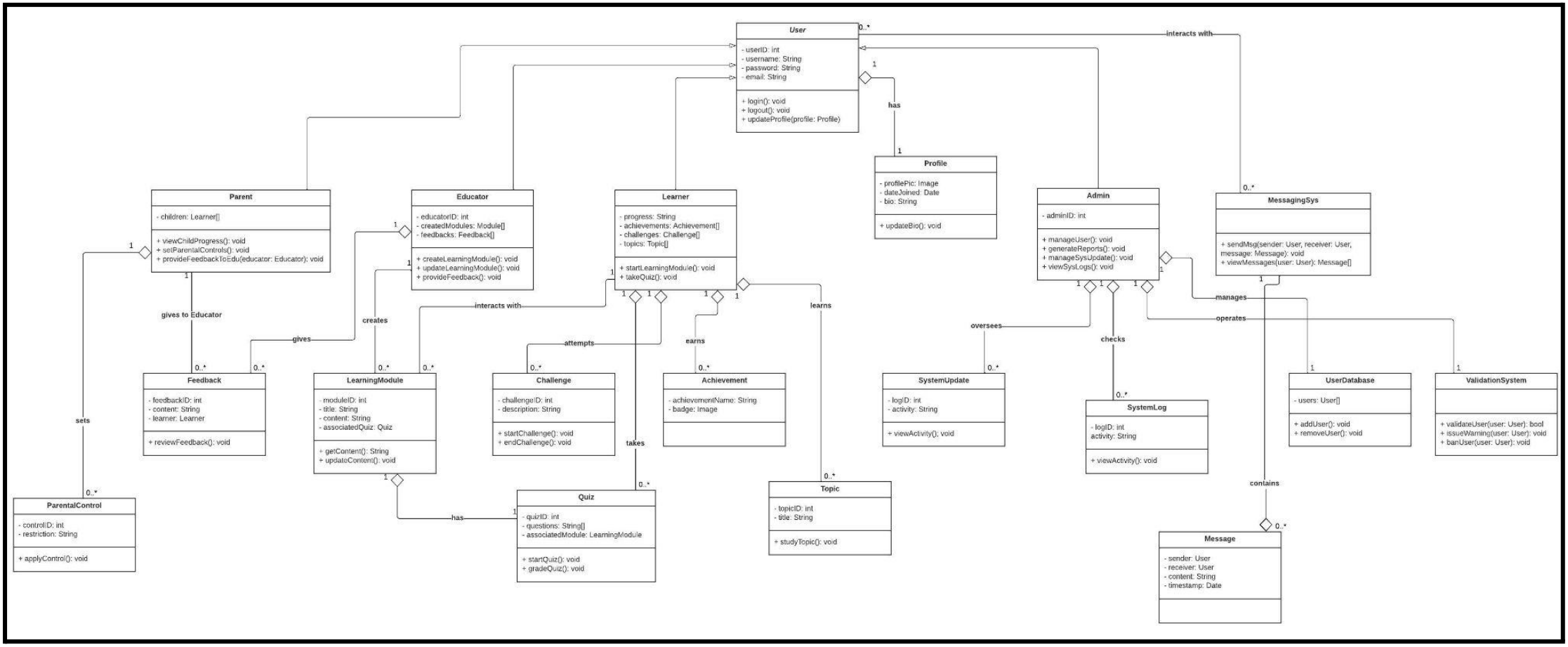


Figure 3.0 – Class Diagram

3.1 Impact of the system architecture decision

Given the Layered Architecture Style we selected, the class diagram (Figure 3.0) reflects the **modular** and **well-organised nature** of our system components.

The **Data Access Layer** directly correlates with the foundational classes, like User, Profile, LearningModule, and others, that dictate the data structures we'll use.

The **Business Logic Layer** influences the methods and associations in the classes. For instance, the interaction of the User with LearningModule, or the way Quiz is associated with both User and LearningModule.

The **Application Layer** manages user sessions, and thus the relationship between the User and their Profile and Progress can be influenced by this layer.

Lastly, the **Presentation Layer** doesn't impact the class structure directly but influences the way data from these classes are presented to end-users.

The architecture decision underpins the modular nature of the classes and their interactions. It ensures that each class can be developed, tested, and maintained separately while still integrating seamlessly with others, reflecting the scalability, modularity, and maintainability objectives of our Layered Architecture Style.

4. Use Case Diagrams

4.1 Learner

- **Register:** Allows new learners to create an account.
- **Login:** Authenticate and access the platform.
- **Choose Learning Module:** Select a module to start or continue learning.
- **Attempt Quiz:** Test knowledge after completing a module or topic.
- **Participate in Challenge:** Engage in coding challenges to reinforce learning.
- **View Progress:** Check learning progress, achievements, and scores.
- **Update Profile:** Modify personal details, preferences, or settings.
- **Provide Feedback:** Share feedback on courses, quizzes, or overall platform.
- **Logout**: Safely exit from the platform.

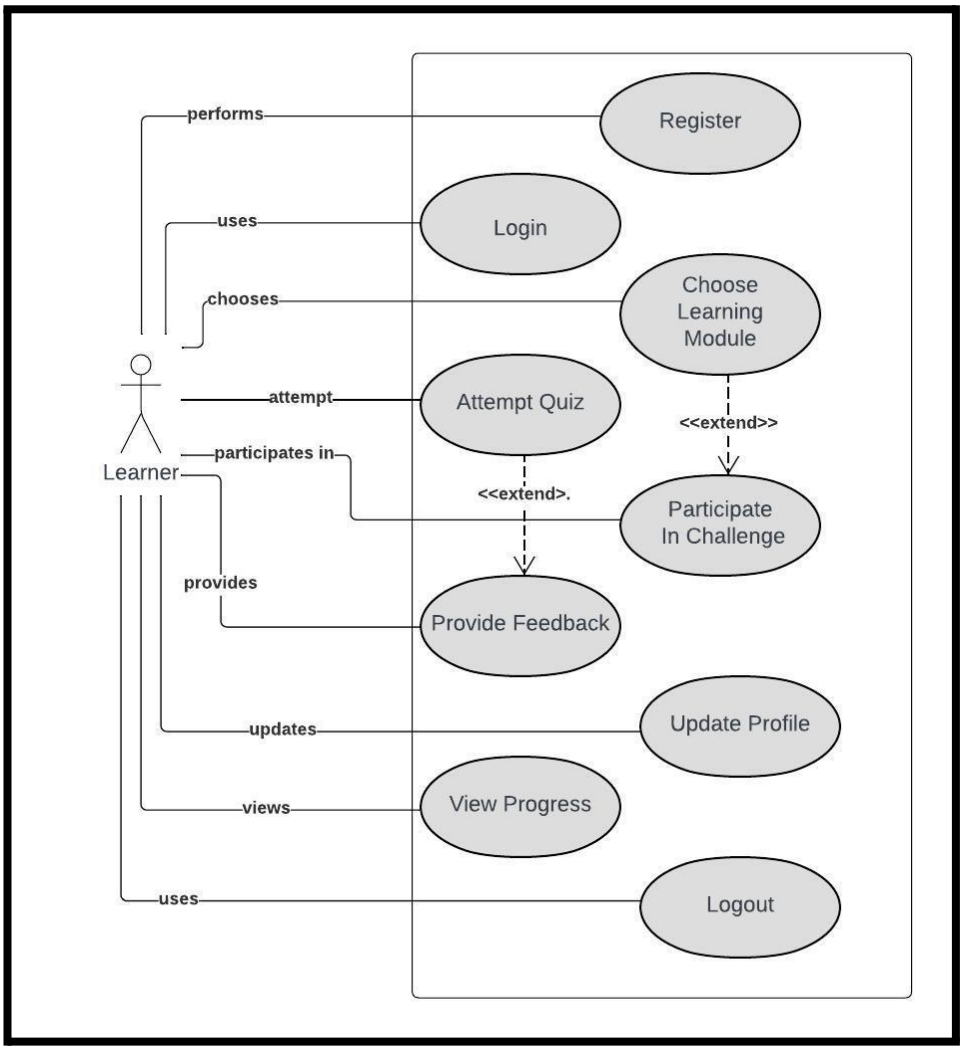


Figure 4.1 - Learner use cases

4.2 Educator

- **Login:** Authenticate and access the educator panel.
- **Upload Learning Module:** Create and share new modules or update existing ones.
- **Review Feedback:** Check feedback provided by learners.
- **Modify Quiz or Challenge:** Update or enhance quiz/challenge content.
- **View Learner's Progress:** Monitor individual or collective learner progress.
- **Interact with Learners:** Respond to queries or assist in learning.
- **Logout:** Safely exit from the educator panel.

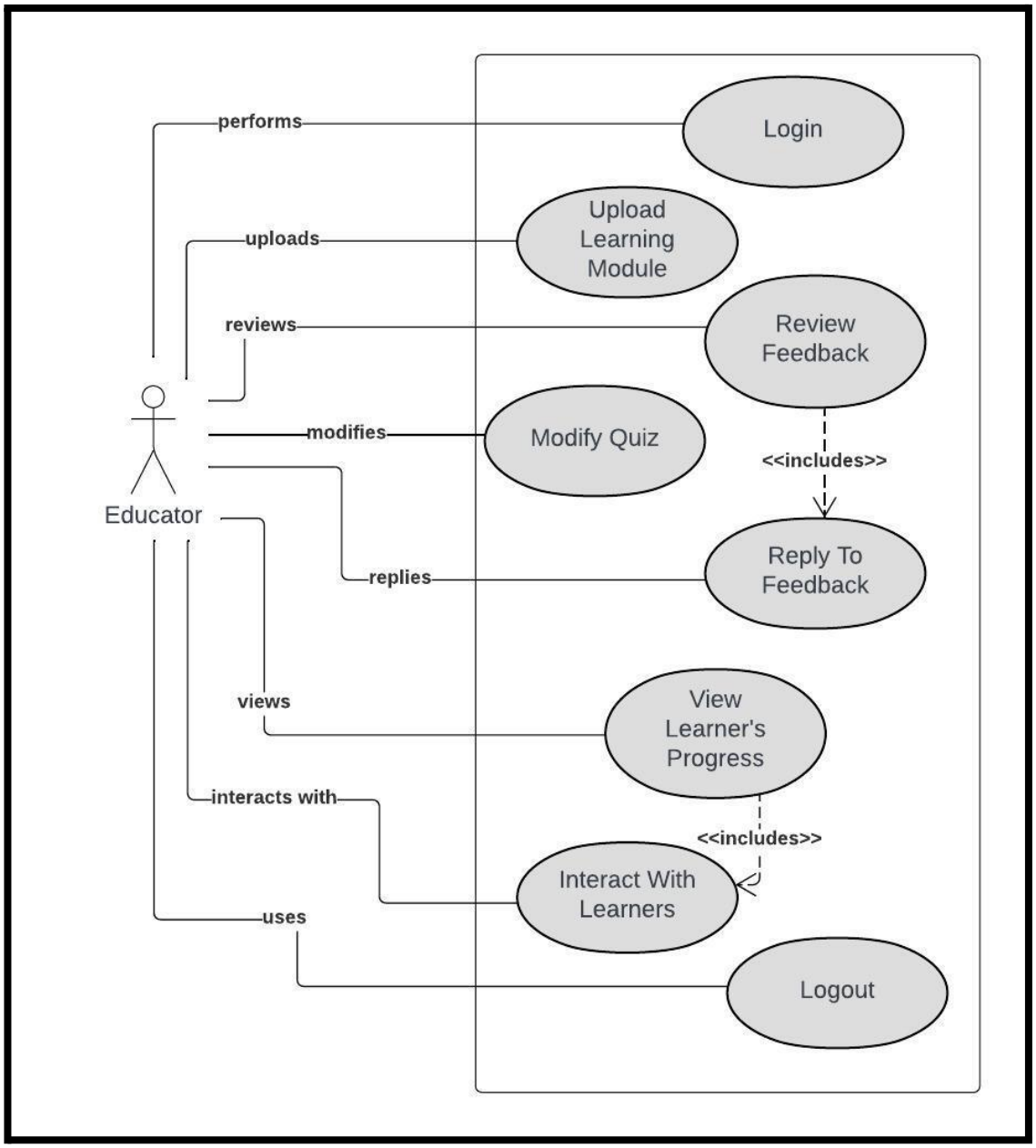


Figure 4.2 - Educator use cases

4.3 Administrator

- **Login:** Authenticate and access the admin dashboard.
- **Manage Users:** Oversee and manage both educator and learner accounts.
- **Moderate Feedback:** Review, approve, or remove feedback.
- **Audit System:** Ensure the platform's performance, security, and reliability.
- **Update System:** Implement new features, fixes, or improvements.
- **Manage Content:** Oversee and approve modules, quizzes, and challenges uploaded by educators.
- **Generate Reports:** Create reports on user activity, system performance, or feedback.
- **Logout:** Safely exit from the admin dashboard.

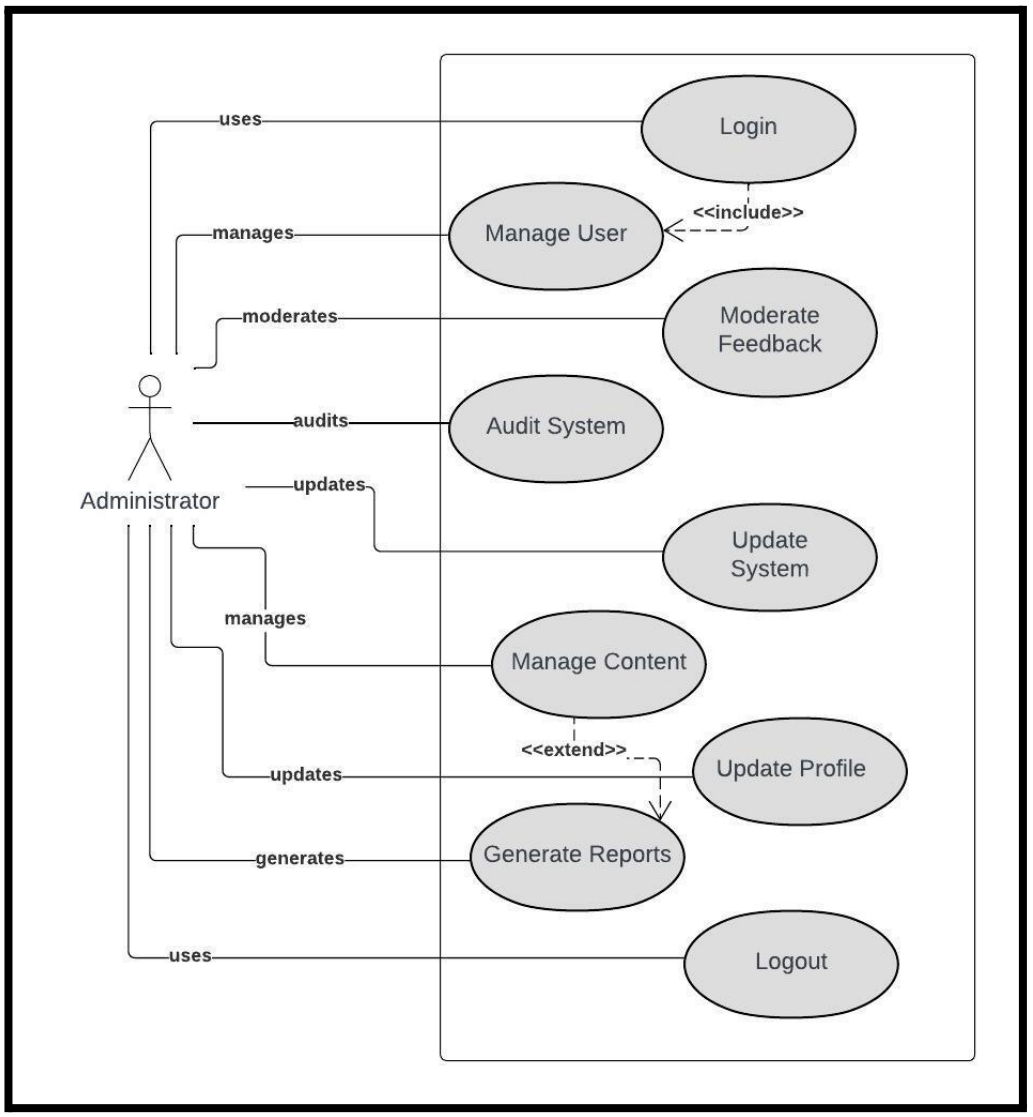


Figure 4.3 - Administrator use cases

4.4 Parent/Guardian

- **Register/Login:** Create an account to monitor a child's progress.
- **View Child's Progress:** Check the learning progress and achievements of a linked child account.
- **Provide Feedback:** Share feedback on courses, content, or overall platform suitability for their child.
- **Interact with Educators:** Raise concerns or queries about learning materials.
- **Logout:** Safely exit from the platform

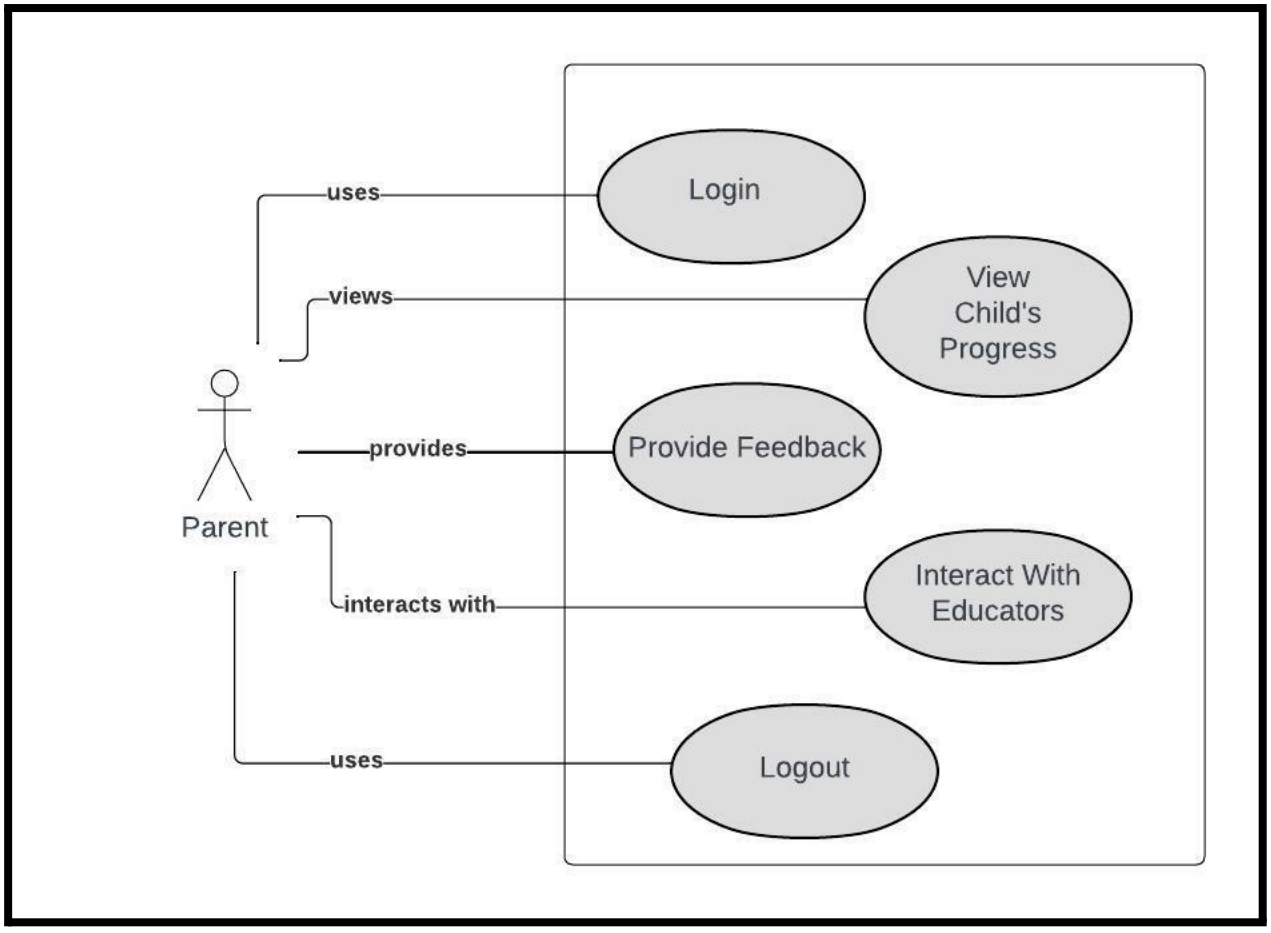


Figure 4.4 - Parent use cases

5. Sequence Diagrams

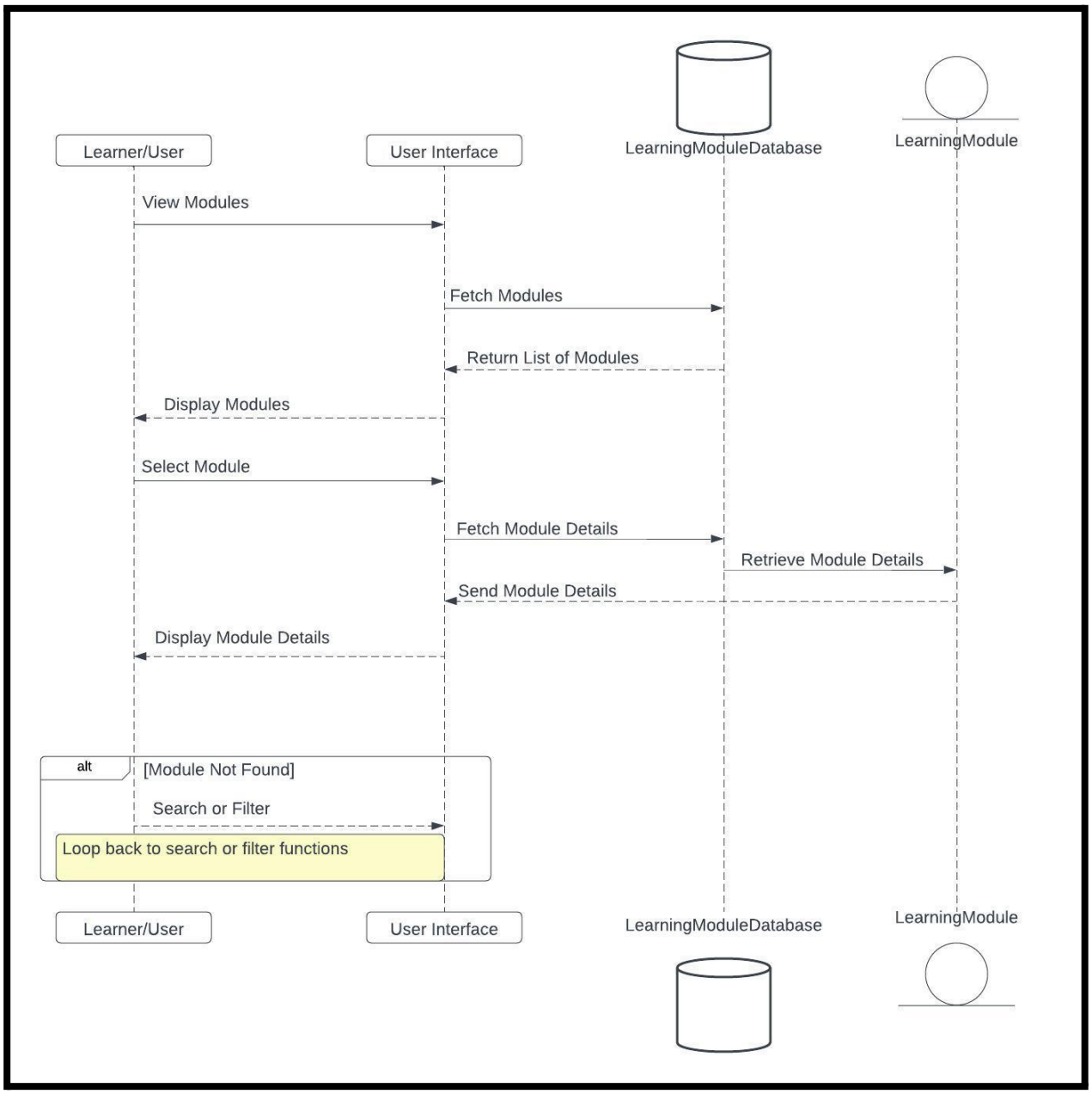


Figure 5.1 - "Choose Learning Module" Sequence Diagram

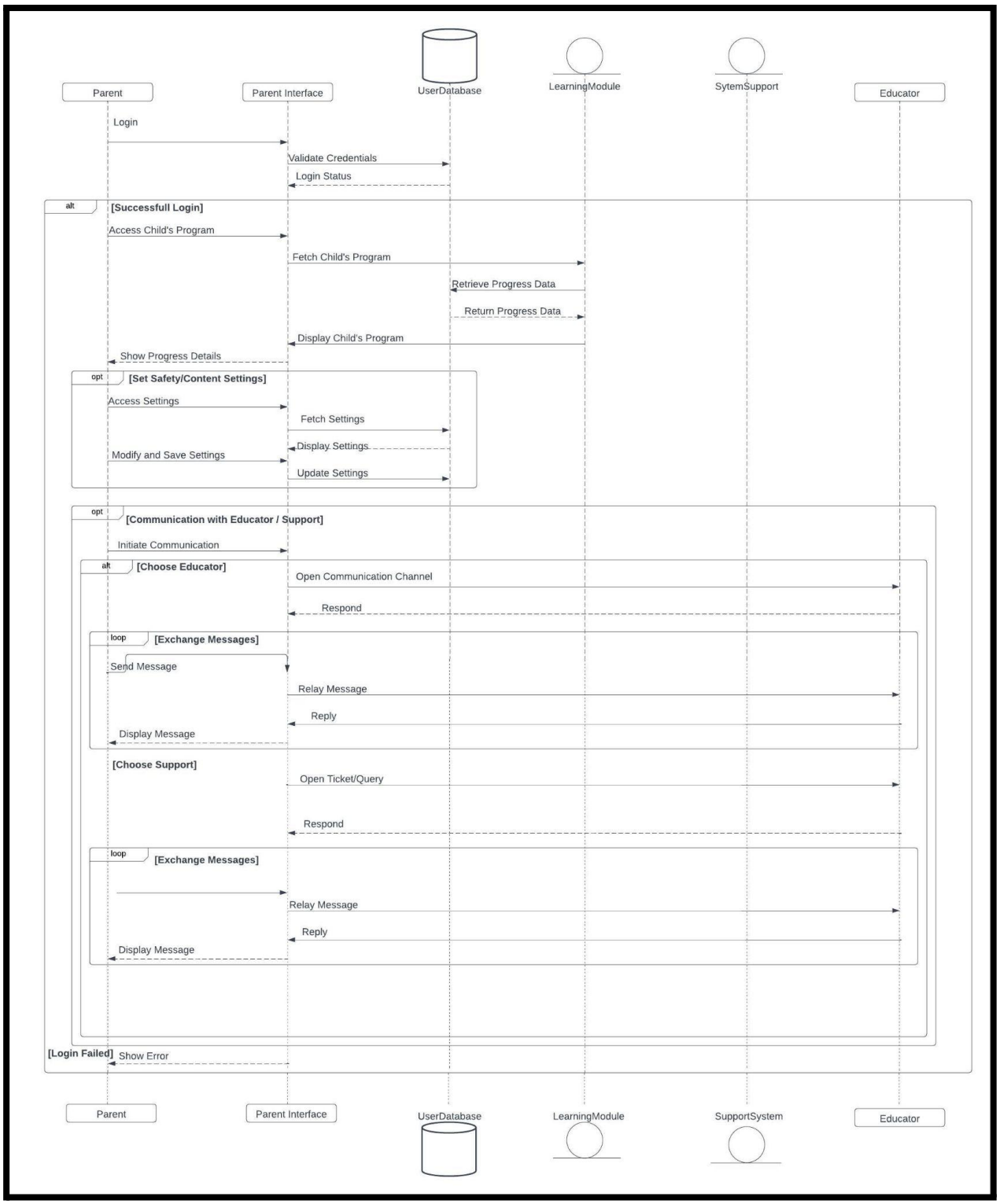


Figure 5.2 - "Parental Control and Messaging" Sequence Diagram

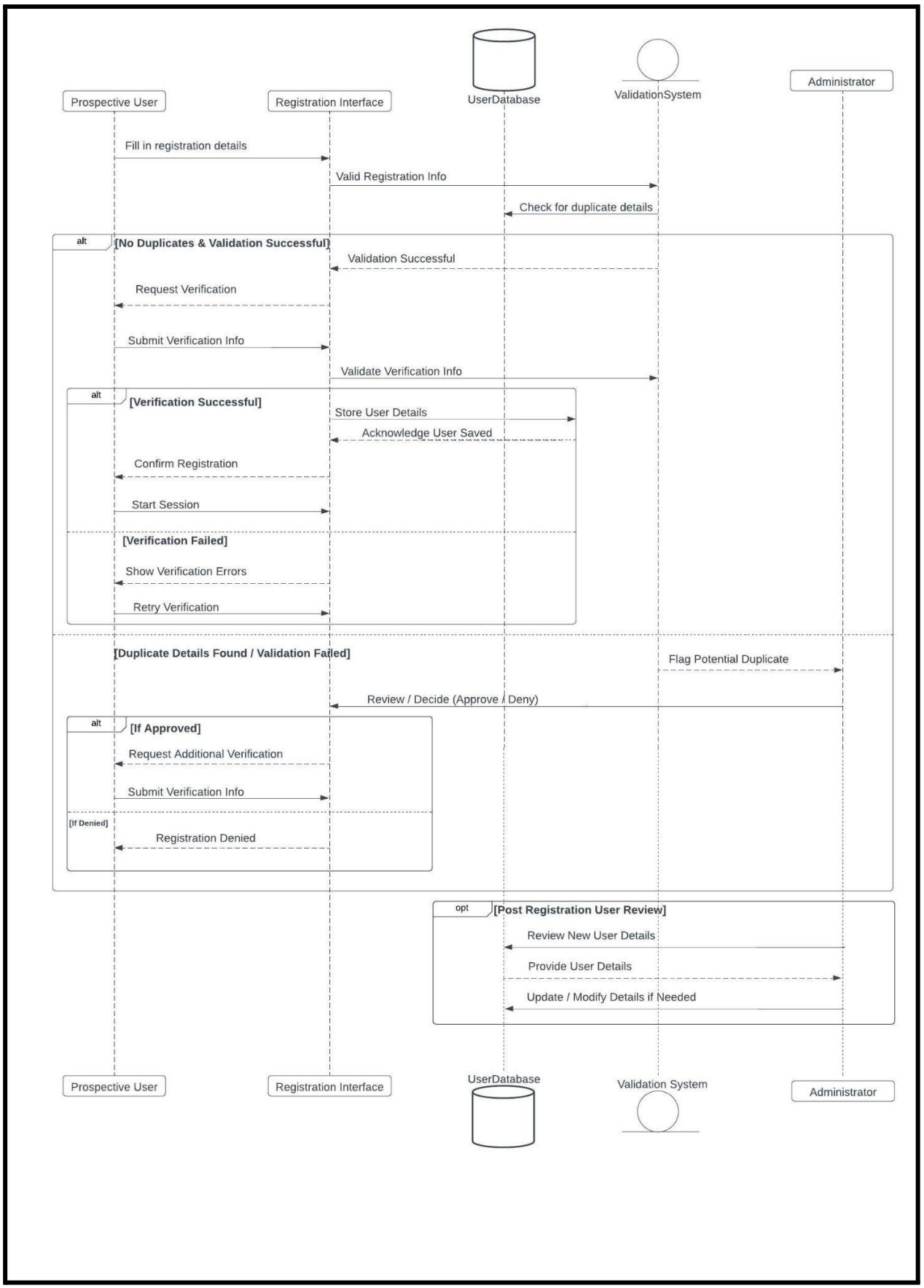


Figure 5.3 - "Administrator Oversees User Registration" Sequence Diagram

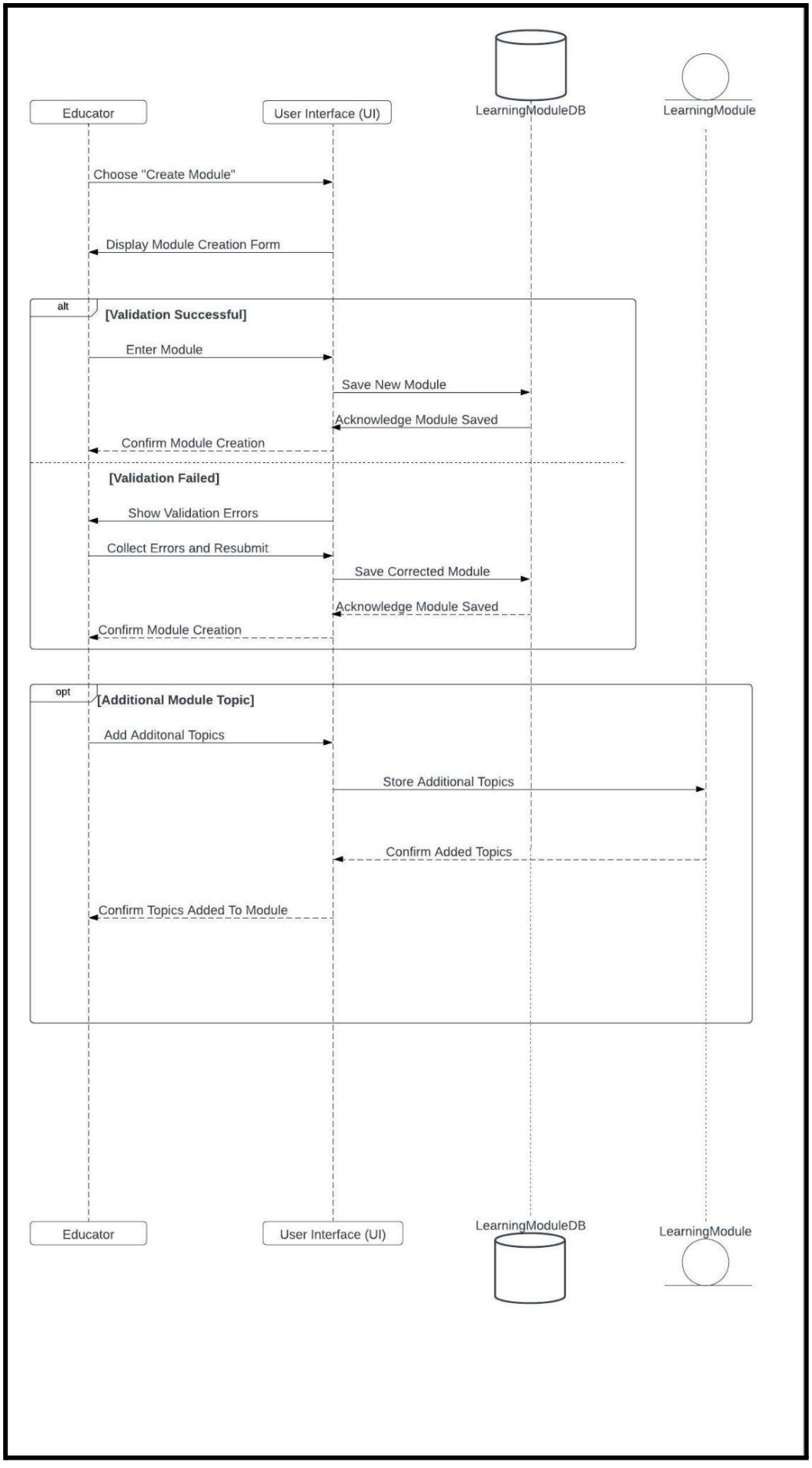


Figure 5.4 - "Educator Creates a New Learning Module" Sequence Diagram